

Scoping of Proposed Dash Projects (July 2026, v2.2)

This document scopes candidate Dash projects against four roadblocks identified for a feature-complete Core, pooled masternodes, speed of capital movement to and from Platform, DashPay usability, and privacy. Two community proposals (pay to a username that is not yet a contact, InstantSend on zero-input transactions) plus a no-UTXO-reuse privacy sketch decompose into seven separable projects. All layers are scoped, Core, Mobile, and Platform, so the portfolio can be chosen deliberately rather than by constraint. Credits are at the end.

Revision note. Version 2 incorporates independent adversarial reviews by multiple AI models, plus primary-source checks against DIP-27, BIP352, and live decentralized-exchange pool data (2026-07-08). The material changes are listed here.

- P1 is restructured around a cheap specification-audit phase, because DIP-27 permits an expired withdrawal to be retried without guaranteeing an identical payout, and that unstated invariant, not quorum authorization, is what blocks instant acceptance today.
- P3 is re-scoped, since BIP352 as specified assumes the Taproot stack (BIP340, BIP341, BIP350), which Dash does not have.
- P4 is restricted to contact-to-contact payments, since splitting a payment breaks single-address invoice flows.
- P2's beneficiary claim is corrected (coinbase payouts also reach masternodes every block, and treasury proposals through superblock coinbases).
- P5's evidence is corrected in both directions, Maya Protocol has live ZEC and DASH pools (verified against pool data), but shielded-address swap support is not verifiably live anywhere, and shielding is the leg the scheme requires.
- P6's extended-public-key risk statement is corrected and its two designs are reordered.
- v2.1 corrects P5 once more after a third-review catch. Shielded ZEC swap support has shipped on Maya Protocol, so the structural claim that shielded notes cannot work in these swap architectures is withdrawn, and the drop argument now rests on the Maya-chain metadata leak and the economics.
- v2.2 adds two design observations from a comparison with Quai's Qi ledger, which runs the same no-reuse model at the protocol level. P4 gains a denominated-payments enhancement, since fixed note sizes coarsen amounts and may make ordinary and mixed payments harder to distinguish, a hypothesis the prototype will test. P7 gains the framing that the Platform credit pool is Dash's native consolidation valve, usable for privacy only if Platform's own records are ever made private.

Summary

#	Project	Layer	Change class	Size	Verdict
P1	Instant acceptance for asset unlock transactions	Core	Spec audit, then policy (A) or quorum messaging (B)	D0 1-2 wk; A: S-M; B: L	Do first, phase-gated

#	Project	Layer	Change class	Size	Verdict
P2	Coinbase maturity relaxation under ChainLocks	Core	Consensus hard fork	S code, L process	Fold into a future hard fork, not standalone
P3	Silent-payment-style static addresses	Core + Mobile	Protocol adaptation + wallet + index infrastructure	L (adapted variant), XL (Taproot path)	Strategic, research gate first
P4	No-input-merging coin selection	Mobile	Wallet policy only	S + measurement	Prototype, contact payments only
P5	Cross-chain shielded consolidation (via ZEC)	Mobile + external	Product integration	L	Drop
P6	Pay to a username that is not yet a contact	Platform + Mobile	Data contract + wallet	M-L	Park until Platform stabilizes, design now
P7	Shielded instant credit withdrawals	Core + Platform	Research-grade	XL	Decompose; long-term home of the consolidation valve

P1. Instant acceptance for asset unlock transactions (Core)

Asset unlock transactions (DIP-27 type-9 special transactions, the credit-withdrawal path from Platform) have zero inputs, derive uniqueness from a withdrawal index, carry an explicit miner-fee field, are signed by a Platform validator quorum (LLMQ_100_67 on mainnet), are ineligible for InstantSend, and are refused once chain height exceeds their signing height by 48 blocks. Finality arrives when the transaction is mined into a ChainLocked block, roughly 2.5 minutes on average with a worse tail, and that is what every exchange and merchant receiving a withdrawal waits for.

The v1 draft framed the fix as an ISLOCK analog. The reviews and a re-read of DIP-27 sharpen the picture. The transaction arriving in the mempool is already quorum-authorized, so authorization is not the gap. The gap is replacement uncertainty. DIP-27 says an expired withdrawal “can be retried” and does not state that the retried transaction preserves the payout script and amount. A recipient crediting on mempool arrival therefore cannot prove the withdrawal it sees is the one that will confirm.

Phase D0, specification and implementation audit (1-2 weeks). Establish whether one withdrawal index binds to exactly one payout (script and amount) across retries, in the DIP text

and in the Platform implementation, and audit the fee-assignment and mempool-eviction behavior of unlock transactions. Everything downstream branches on the answer.

- Path A, the invariant holds or can be pinned with a spec clarification. Instant acceptance becomes receiving-wallet and integrator policy, credit a valid quorum-signed type-9 transaction on mempool arrival, plus published exchange guidance. No new network messages. Residual risks are non-inclusion and eviction, both bounded by the retry mechanism once the payout invariant is guaranteed. Size S-M.
- Path B, the invariant does not hold, or attested finality is wanted for SPV clients and for child-transaction chaining. A deterministic lock message binds index, txid, and payout. Unlike DIP-10 locks, which never expire, this lock must expire with the transaction it covers, and the design must specify behavior when a conflicting unlock is mined and ChainLocked, an explicit eligibility-rule change so children spending locked unlock outputs can receive ordinary ISLOCKS (they do not qualify under current DIP-10 rules), quorum signing load and denial-of-service limits, persistence, and the P2P and RPC surface, all through the DIP process. Size L.

The v1 claim that this is “the same character of change as DIP-10” is withdrawn. Path B creates a new conflict domain with its own expiry, retry, and reorg rules, and its delivery cost includes the DIP process, tests, deployment, and integrator rollout. The project keeps its priority because D0 is cheap, both paths attack the capital-movement roadblock directly, and Path A may deliver most of the value at a fraction of the cost.

P2. Coinbase maturity relaxation under ChainLocks (Core)

InstantSend is the wrong lever for coinbase transactions, which exist only once mined. The barrier to spending them is the 100-block maturity rule, which protects against reorgs orphaning the reward. ChainLocks (DIP-8) prevent deep reorgs, so a rule such as “coinbase spendable once its block is ChainLocked” is coherent in outline. Two corrections from review. The beneficiaries are broader than v1 stated. Every block’s coinbase pays the scheduled masternode payee alongside the miner under consensus enforcement, and treasury proposals are paid in superbloc coinbases, so masternode operators and funded proposals face the same maturity delay, which improves the governance case. And the design is harder than v1 implied, because ChainLock signatures are not committed in blocks and old signatures are not necessarily available during initial block download, so a consensus rule referencing ChainLock state needs a concrete answer for how a syncing node validates historical spends. Still a hard fork whose process cost dominates its small code cost. Recommendation unchanged, keep it drafted with the initial-sync question answered and attach it to whichever hard fork ships next.

P3. Silent-payment-style static addresses (Core + Mobile)

The goal stands, a single static payment identifier that senders convert into a fresh on-chain address with no interaction, no notification transaction, and no address reuse, serving the DashPay-usability and privacy roadblocks with nothing needed from Platform. The v1 scoping as a “BIP352 port” was wrong. BIP352 as specified assumes Schnorr signatures (BIP340), Taproot outputs (BIP341), and bech32m addresses (BIP350), none of which exist on Dash.

Two paths.

- Taproot path: bring the Schnorr and Taproot stack to Dash first, including a new bech32m address format, then port BIP352. A multi-release consensus program. Size XL, and not a mid-term project.
- Adapted path: redesign the derivation for ECDSA and P2PKH, with the recipient publishing scan and spend public keys and the sender deriving a Diffie-Hellman-tweaked public key hashed to a standard pay-to-public-key-hash address. Cryptographically this is the BIP47 primitive without the notification transaction, and P2PKH inputs expose the sender public keys the derivation needs. One review claimed a non-Taproot construction would be immediately identifiable on-chain. That applies to a new output type, not to this construction, since a tweaked P2PKH output is byte-identical to any other P2PKH output. The real costs are a Dash-specific DIP with security review of the adapted derivation, no reuse of upstream BIP352 tooling, and the unchanged scanning economics, full receive-side scanning for full nodes and tweak-index servers for SPV mobile clients. Size L.

Gate either path on a two-part research memo, the adapted derivation specification and a scan-cost benchmark on realistic Dash transaction volume. No build commitment before the memo.

P4. No-input-merging coin selection (Mobile, contact payments only)

A wallet-side coin-selection rule that never combines inputs from different addresses, paying 11 from a 10 and a 1 as two transactions to two recipient addresses. No protocol change. It defeats the common-input-ownership heuristic, the workhorse of chain-analysis clustering.

Review added a scope restriction that v1 missed. Splitting a payment requires multiple recipient addresses and breaks single-address flows, BIP21 invoices, payment processors, and exchange deposit addresses all present one destination and one expected amount, and a split payment can fail invoice matching and refunds. The policy is therefore only viable for contact-to-contact DashPay payments, where DIP-15 key exchange already lets the sender derive as many recipient addresses as needed, and it must never apply to invoice or deposit flows.

The privacy gain also remains partial, since the split transactions broadcast together and land in one wallet, so timing and amount correlation re-link them with decent confidence, and fees roughly double per payment. A comparison with Quai's Qi ledger, which enforces the same no-reuse model at the protocol level, adds a cheap enhancement worth prototyping alongside. Qi permits only sixteen fixed note denominations, which coarsens amounts so that payment size stops being a fine-grained fingerprint, though denomination vectors, output counts, and timing still leak structure. Dash CoinJoin already defines standard denominations, so a no-merge wallet could prefer CoinJoin-standard denominations for its outputs, aiming at the stated goal of privacy that does not present as a flagged feature. Whether denominated no-merge payments actually become hard to tell apart from CoinJoin activity is a hypothesis for the prototype to test, since CoinJoin rounds, denomination creation, and ordinary sends have different transaction shapes. The cost is more outputs per payment, which folds into the same fee and UTXO measurements. The v1 claim that every payment necessarily mints more UTXOs than it consumes was too strong, the net effect depends on change outputs and later consolidation. The right scope is a Dash Mobile prototype behind a toggle, active only for contact payments, with five measured outputs, fee overhead per payment, net UTXO growth under simulated adoption, re-linkage rate from a timing-and-amount correlator standing in for a chain-analysis adversary, distinguishability of denominated no-merge payments from CoinJoin activity, and zero regressions on invoice flows. Ship or kill on those numbers.

P5. Cross-chain shielded consolidation (drop)

The sketch sends fragmented UTXOs to the Zcash Orchard pool and back at coordinated intervals. Dash has no shielded pool, so this means a round trip, DASH to ZEC to DASH, over a cross-chain decentralized exchange. The evidence base is corrected from v1 in both directions. Maya Protocol has live ZEC and DASH pools, both with status “available” in its Midgard pool data (<https://midgard.mayachain.info/v2/pools>, queried 2026-07-08), and THORChain announced a phased ZEC rollout in June 2026 that was not verifiable against live pool data at review time. Shielded-address handling in the swap path has also shipped on Maya, wallet integrations and contributor documentation describe live shielded ZEC swap legs, with the protocol’s observers using Zcash viewing keys to decrypt and verify the shielded outputs they handle. That support does not rescue the scheme, because the privacy stops at the Zcash leg. Every swap is recorded on the Maya chain itself with addresses and amounts in clear, so a consolidation round trip publishes its intent, size, and timing on a third public ledger even when the ZEC leg is shielded, and amount and timing correlation reconnect the DASH ends across it.

The economic and correlation objections stand independently of listing status. Each user pays swap fees and slippage twice and carries ZEC price exposure during the trip. Amounts correlate in and out unless standardized into denominations, at which point the design has rebuilt mixing with extra steps. Coordinated consolidation intervals are a timing fingerprint and concentrate demand against limited pool liquidity. Compliance desks treat shielded-ZEC-adjacent flows the way they treat mixers, so the optics motivation fails. And the correct baseline is Dash’s own CoinJoin, which ships in Dash Core with mobile-wallet support following, so any consolidation-privacy proposal has to beat CoinJoin on cost, liquidity, and compliance treatment, which this does not. Recommendation unchanged, drop this leg and let P4’s measurements decide whether fragmentation is a real problem at realistic adoption.

P6. Pay to a username that is not yet a contact (Platform + Mobile)

Usernames live in the Dash Platform Name Service and payment derivation lives in the DashPay contract (DIP-15), where mutually accepted contact requests exchange encrypted extended public keys. Paying a non-contact therefore requires the recipient to pre-publish key material. The v1 statement that third-party address derivation is “harmless” was too casual. Publishing a plain extended public key exposes every address on that derivation branch, enabling surveillance of the branch’s incoming payment graph, dusting, and address probing, and imposing a scanning burden on the recipient. Any design must publish derivation material that does not grant branch-wide watching.

Two viable designs, reordered from v1 because the silent-payment variant now depends on P3’s research gate.

- BIP47-style (implementable today): add a public payment-code field to the DashPay profile document. The sender derives a one-off P2PKH address from their identity key and the recipient’s payment code, and posts a state transition as the notification telling the recipient which derivation to watch. Platform credit fees rate-limit notification spam. One caveat the reviews added, the notification documents sit on Platform, and even with encrypted payloads the sender-to-recipient graph edge may be visible to Platform observers, so the document and query model need a privacy review.

- Silent-payment-style (depends on P3): publish the P3 identifier in the profile instead. No notification is needed, and Platform serves purely as a name-to-identifier directory, so payments remain valid even if Platform is unavailable, which contains the instability risk to lookup rather than to funds.

Sizing is M for the BIP47-style variant and inherits P3's L for the other. Both wait on Platform stability for implementation. Writing the design document now is cheap and worthwhile.

P7. Shielded instant credit withdrawals (decompose)

The stated end goal, an instantly final and shielded withdrawal from Platform, bundles two projects of very different maturity. The instant half is P1 and is tractable now. The shielded half remains research-grade, but v2.2 records a framing that gives it a concrete purpose. The Platform credit pool is structurally the consolidation valve that the P5 sketch went looking for on other chains. Asset lock transactions deposit fragmented UTXOs into a pooled balance, and withdrawals return as zero-input asset unlock transactions with no UTXO-graph link to the deposits, the same shape as Quai's protocol-level conversion between its UTXO ledger and its account ledger (a shape analogy only, since Quai's conversions are transparent and its consolidation privacy instead comes from protocol-incentivized cooperative reaggregation, a many-party CoinJoin-like mechanism). Today a credit pool round trip is not a privacy tool, because the linkage moves to public records, Platform tracks identity balances and the top-up and credit-withdrawal state transitions while the linked Core asset lock carries the deposited amount, so amounts and timing are visible on both legs, the same metadata weakness that sank the cross-chain route in P5. It becomes one only if the Platform-side records are made private, which is the shielded half of this project and a multi-year research question, not a scopeable feature. The dependency order is concrete, P1 makes pool round trips fast enough to be usable at all, P4 and P3 carry the near-term privacy roadmap, and P7 is the eventual home of consolidation privacy if the research half lands. Recommendation unchanged in substance, ship P1 and revisit the shielded half only as a seriously resourced research proposal.

Recommended sequence

1. P1 phase D0 now, the 1-2 week specification audit of the retry-payout invariant, then Path A (policy and integrator guidance) or Path B (lock message through the DIP process) as the audit dictates.
2. P4 in parallel, Mobile prototype behind a toggle restricted to contact payments, with the five measurements as the ship-or-kill gate.
3. P3 research memo next, the adapted ECDSA derivation specification plus the scan-cost benchmark, before any build commitment.
4. P6 as a design document now, implementation once Platform stabilizes, BIP47-style variant first unless the P3 memo lands.
5. P2 folded into the next hard fork once the initial-sync question is answered, P5 dropped, P7 reduced to P1 plus the privacy track.

Credits

The originating proposals and the four-roadblock framing are due to Joel Valenzuela (TheDesert-Lynx), whose ideas on paying usernames that are not yet contacts, extending InstantSend to

zero-input transactions, and wallet-level UTXO hygiene seeded this document. The scoping, the verdicts, and any errors are the author's.

The claims in this document were adversarially reviewed by multiple independent AI models over several rounds, and every disputed claim was adjudicated against primary sources, the dashpay/dips and bitcoin/bips repositories, Dash Core source, and live decentralized-exchange pool data.